

Transformari in PySpark

Transformari pe DataFrame

Pentru exemplificare, se vor crea 3 data DataFrame-uri

Employee Data (df1)

```
# employee data
data = [
    ["1", "sravan", "company 1"],
    ["2", "ojaswi", "company 1"],
    ["3", "rohith", "company 2"],
    ["4", "sridevi", "company 1"],
    ["5", "bobby", "company 1"]
]

cols = ['ID', 'NAME', 'Company']

df1 = spark.createDataFrame(data, cols)
df1.show()
```

Rezultat:

```
+---+-----+-----+
| ID| NAME  | Company|
+---+-----+-----+
| 1 |sravan |company 1|
| 2 |ojaswi |company 1|
| 3 |rohith |company 2|
| 4 |sridevi|company 1|
| 5 |bobby  |company 1|
+---+-----+-----+
```

Salary Data (df2)

```
# salary data
data1 = [
    ["1", "45000", "IT"],
    ["2", "145000", "Manager"],
    ["6", "45000", "HR"],
    ["5", "34000", "Sales"]
]

cols1 = ['ID', 'salary', 'department']

df2 = spark.createDataFrame(data1, cols1)
df2.show()
```

Rezultat:

```
+---+-----+-----+
| ID|salary|department|
+---+-----+-----+
```

```
| 1 |45000 |IT      |
| 2 |145000|Manager  |
| 6 |45000 |HR       |
| 5 |34000 |Sales    |
+---+-----+-----+

```

Location Data (df3)

```
# location data
data2 = [
    ["1", "Bucharest"],
    ["2", "Cluj"],
    ["3", "Timisoara"],
    ["5", "Iasi"],
    ["7", "Brasov"]
]

cols2 = ["ID", "location"]

df3 = spark.createDataFrame(data2, cols2)
df3.show()
```

Rezultat:

```
+---+-----+ | ID| location | +---+-----+ | 1 |Bucharest| | 2 |Cluj
| | 3 |Timisoara| | 5 |Iasi | | 7 |Brasov | +---+-----+

```

select()

- Selecteaza coloane specifice sau creezi coloane noi cu expresii.

```
# selectam doar numele si compania
df1.select("NAME", "Company").show()

# sau adaugam o coloana noua calculata
from pyspark.sql.functions import col, lit

df1.select(col("NAME"), (lit("Employee")).alias("Role")).show()
```

withColumn()

- Creeza sau modifica o coloana existenta.

```
from pyspark.sql.functions import upper

# adaugam o coloana cu nume mare
df1.withColumn("NAME_UPPER", upper(col("NAME"))).show()
```

filter() / where()

- Filtreza randurile dupa o conditie.

```
# selectam angajatii cu ID < 4
df1.filter(col("ID") < "4").show()
df1.where(col("Company") == "company 1").show()
```

drop()

- Sterge o coloana.

```
df1.drop("Company").show()
```

distinct()

- Elimini duplicate.

```
df2.distinct().show()
```

groupBy() + agg()

- Agregari (count, sum, avg, max, min) pe grupuri.

```
df2.groupBy("department").agg({"salary": "avg"}).show()
```

orderBy() / sort()

- Sortezi randurile.

```
df2.orderBy(col("salary").desc()).show()
```

join()

- Join intre DataFrame-uri (inner, left, right, full, semi, anti).

```
df1.join(df2, "ID", "inner").show()
df1.join(df3, "ID", "left").show()
```

union()

- Imbina doua DataFrame-uri cu aceleasi coloane.

```
df1_subset = df1.select("ID", "NAME")
df2_subset = df2.select(col("ID"), col("department").alias("NAME"))
df_union = df1_subset.union(df2_subset)
df_union.show()
```

`withColumnRenamed()`

- Schimba numele unei coloane.

```
df1.withColumnRenamed("NAME", "Employee_Name").show()
```

Funcții uzuale din `pyspark.sql.functions`

- `col()`, `lit()` – pentru referire la coloane sau valori constante
- `upper()`, `lower()`, `trim()` – manipulare string
- `concat()`, `concat_ws()` – concatenare string
- `when()`, `otherwise()` – condiții tip if/else
- `round()`, `ceil()`, `floor()` – operații numerice
- `date_format()`, `current_date()`, `datediff()` – operații pe date

Exemplu combinat:

```
from pyspark.sql.functions import when, concat_ws
```

```
df1.withColumn("Seniority", when(col("ID") < "3", "Junior").otherwise("Senior"))
```

```
.withColumn("FullInfo", concat_ws(" - ", col("NAME"), col("Company")))
```

```
.show()
```

```
from pyspark.sql.functions import when, concat_ws, col

df1.withColumn(
    "Seniority",
    when(col("ID") < "3", "Junior").otherwise("Senior")
).withColumn(
    "FullInfo",
    concat_ws(" - ", col("NAME"), col("Company"))
).show()
```

`from pyspark.sql.functions import when, concat_ws, col`

- `when` → este o funcție pentru condiții tip **if/else** în DataFrame.
- `concat_ws` → concatenează mai multe coloane sau valori într-un **string**, cu un separator specificat. (ws = with separator)
- `col` → permite referirea la o coloană existentă în DataFrame.

`withColumn("Seniority", when(...).otherwise(...))`

- `withColumn` creează sau modifică o coloană.
- `"Seniority"` → numele noii coloane.
- `when(col("ID") < "3", "Junior")`
 - verifică fiecare rând dacă valoarea din coloana ID este mai mică decât "3".
 - dacă da, pune valoarea "Junior" în coloana Seniority.
- `.otherwise("Senior")`
 - dacă condiția nu este adevărată, pune "Senior".

Practic: toate ID-urile 1 si 2 vor fi "Junior", restul "Senior".

withColumn("FullInfo", concat_ws(" - ", col("NAME"), col("Company")))

- Creeaza o noua coloana "FullInfo".
- concat_ws(" - ", col("NAME"), col("Company"))
 - ia valorile din coloanele NAME si Company.
 - le uneste intr-un string, separate prin " - ".
- Ex: pentru primul rand, "sravan" si "company 1" devine "sravan - company 1".

```
+-----+-----+-----+-----+-----+
| ID| NAME | Company |Seniority| FullInfo          |
+-----+-----+-----+-----+-----+
| 1 |sravan |company 1| Junior  | sravan - company 1|
| 2 |ojaswi |company 1| Junior  | ojaswi - company 1|
| 3 |rohith |company 2| Senior  | rohith - company 2|
| 4 |sridevi|company 1| Senior  | sridevi - company 1|
| 5 |bobby  |company 1| Senior  | bobby - company 1 |
+-----+-----+-----+-----+-----+
```

Rezultat

Tipuri de Join in PySpark – Unirea a doua DataFrame-uri

In PySpark, operatiile de tip **join** sunt folosite pentru a combina randuri din doua DataFrame-uri pe baza unei **coloane comune**. Exista mai multe tipuri de join, precum **inner**, **left**, **right**, **full outer**, **left semi** si **left anti**, fiecare fiind utilizat in functie de modul in care dorim sa tratam randurile care se potrivesc sau nu intre cele doua seturi de date.

Sintaxa generala pentru un join este:

```
dataframe1.join(dataframe2, dataframe1.column_name == dataframe2.column_name, "type")
```

unde:

- dataframe1 reprezinta primul DataFrame
- dataframe2 reprezinta al doilea DataFrame
- column_name este coloana folosita pentru potrivirea datelor in ambele DataFrame-uri
- type indica tipul de join care va fi aplicat

INNER JOIN

- Returneaza doar randurile care au valori potrivite in ambele DataFrame-uri.

```
df_inner = df1.join(df2, df1.ID == df2.ID, "inner")
df_inner.show()
```

Rezultat asteptat:

```
+---+-----+-----+-----+-----+
| ID| NAME  | Company |salary|department|
+---+-----+-----+-----+-----+
| 1 |sravan |company 1|45000 |IT         |
+---+-----+-----+-----+-----+
```

2	ojaswi	company 1	145000	Manager
5	bobby	company 1	34000	Sales

LEFT JOIN (LEFT OUTER JOIN)

- Pastreaza toate randurile din DataFrame-ul din stanga, adauga valori din dreapta unde se potrivesc; null daca nu exista potrivire.

```
df_left = df1.join(df3, df1.ID == df3.ID, "left")
df_left.show()
```

Rezultat asteptat:

ID	NAME	Company	location
1	sravan	company 1	Bucharest
2	ojaswi	company 1	Cluj
3	rohith	company 2	Timisoara
4	sridevi	company 1	null
5	bobby	company 1	Iasi

RIGHT JOIN (RIGHT OUTER JOIN)

- Pastreaza toate randurile din DataFrame-ul din dreapta, adauga valori din stanga unde se potrivesc.

```
df_right = df1.join(df3, df1.ID == df3.ID, "right")
df_right.show()
```

Rezultat asteptat:

ID	NAME	Company	location
1	sravan	company 1	Bucharest
2	ojaswi	company 1	Cluj
3	rohith	company 2	Timisoara
5	bobby	company 1	Iasi
7	null	null	Brasov

FULL OUTER JOIN

- Pastreaza toate randurile din ambele DataFrame-uri, null daca nu exista potrivire.

```
df_full = df1.join(df3, df1.ID == df3.ID, "outer")
df_full.show()
```

Rezultat asteptat:

ID	NAME	Company	location
1	sravan	company 1	Bucharest
2	ojaswi	company 1	Cluj
3	rohith	company 2	Timisoara
4	sridevi	company 1	null
5	bobby	company 1	Iasi
7	null	null	Brasov

LEFT SEMI JOIN

- Returneaza randurile din stanga care au potrivire in dreapta, fara coloanele din dreapta.

```
df_left_semi = df1.join(df2, df1.ID == df2.ID, "left_semi")
df_left_semi.show()
```

Rezultat asteptat:

ID	NAME	Company
1	sravan	company 1
2	ojaswi	company 1
5	bobby	company 1

LEFT ANTI JOIN

- Returneaza randurile din stanga care NU au potrivire in dreapta.

```
df_left_anti = df1.join(df2, df1.ID == df2.ID, "left_anti")
df_left_anti.show()
```

Rezultat asteptat:

ID	NAME	Company
3	rohith	company 2
4	sridevi	company 1

Observatii practice

- **Inner join** → pentru potriviri exacte.
- **Left / Right / Full outer** → cand vrei sa pastrezi si randurile fara potrivire.

- **Left semi** → verifici dacă există în alt DataFrame.
- **Left anti** → elimini ce există în alt DataFrame.
- Ordinea join-urilor și numărul de partiții poate influența performanța.

Spark SQL folosește același motor de execuție ca DataFrame API (Catalyst + Tungsten)

- Atât **DataFrame API**, cât și **Spark SQL** folosesc **același motor intern de execuție**.
- **Catalyst optimizer**:
 - Este motorul care optimizează interogările.
 - Decide cel mai eficient mod de a executa join-uri, filtre sau agregări.
 - Ex: dacă faci `df1.join(df2, ...)` sau `spark.sql("SELECT ... FROM df1 JOIN df2")`, Catalyst creează automat **planul fizic optimizat**.
- **Tungsten engine**:
 - Optimizează execuția la nivel de memorie și CPU.
 - Reduce costurile de serializare și deserializare, folosind cod generat dinamic.

Practic: indiferent dacă se folosește SQL sau DataFrame API, Spark procesează datele **aproape la fel de rapid**.

Performanța este echivalentă între SQL și DataFrame joins

- Aceeași operație de join făcută în **PySpark DataFrame API** sau în **Spark SQL** va avea performanță foarte apropiată.
- De ce?
 - Ambele sunt transformate de **Catalyst** într-un **plan fizic optimizat**.
 - Join-urile sunt procesate **distribuit și paralel** pe întreg clusterul.
- Singura diferență reală este **claritatea codului și lizibilitatea**.

SQL este mai clar pentru logica complexă cu mai multe join-uri

- Dacă ai **3, 4 sau mai multe DataFrame-uri** și join-uri diferite (inner, left, right, semi, anti):
 - **SQL** este mai intuitiv și mai ușor de urmărit:
 - Ordinea join-urilor
 - Tipul fiecărui join
 - Condițiile de potrivire
- Exemplu: folosind **CTE-uri (WITH clause)**, poți sparge logica în pași intermediari, ceea ce este mai greu de realizat cu DataFrame API fără multe variabile intermediare.

Poți converti ușor între SQL și DataFrame API

- Orice DataFrame poate fi **înregistrat ca view temporar** și folosit în SQL:

```
df.createOrReplaceTempView("my_table")
spark.sql("SELECT * FROM my_table WHERE ...").show()
```

- La fel, orice query SQL poate fi scris cu DataFrame API:

```
df1.join(df2, df1.ID == df2.ID, "inner") \
    .join(df3, df1.ID == df3.ID, "left") \
    .select(df1.ID, df1.NAME, df2.salary, df3.location)
```

- **Avantaje:**
 - SQL → mai clar și mai ușor de citit pentru BI sau analize complexe.
 - DataFrame API → mai potrivit pentru transformări programatice în Python/Scala (`map`, `filter`, `withColumn`).

Rezumat

- SQL și DataFrame API **nu diferă la performanță** – ambele folosesc **Catalyst + Tungsten**.
- SQL este mai **citibil** pentru join-uri multiple sau logica complexă.
- DataFrame API este mai potrivit pentru **procesări programatice și funcționale**.